PharmaTrace: Technical Report

Tamunotonye Briggs · BSc Software Engineering, African Leadership University Rwanda · Supervisor: Neza David Tuyishimire · 2 November 2026

Repository: github.com/TammyBriggs/PharmaTrace · The walkthrough video link is in the repository README.

PharmaTrace is a decentralized drug verification system for Rwanda. A Flutter app lets patients and pharmacists scan a pack and receive an authenticity verdict. A React dashboard lets the Rwanda FDA register manufacturers and approve, flag or revoke batches. A Node.js API with PostgreSQL connects both to an Ethereum smart contract. Everything runs on a local Hardhat chain, as the approved proposal specifies.

1. Scope alignment and feature validation

The table maps the approved proposal (Chapters 1 and 3) to the delivered product. Evidence paths are relative to the repository root.

Planned feature	Intended purpose	Status	Evidence
Scan a QR code without a wallet and see the batch history	Let patients and pharmacists check a pack in seconds	**Complete**	`pharmatrace_mobile/lib/features/scan`, `verification/batch_history_screen.dart`, `GET /api/verify` and `/api/verify/history`
Regulator registers manufacturers	Only licensed makers may submit batches	**Complete**	`pages/Manufacturers.tsx`; contract `registerManufacturer`
Regulator approves batches	Separate submit and approve roles	**Complete**	`components/BatchActions.tsx`; `batchController.confirmAction`; contract `approveBatch`
Flag counterfeit or suspicious batches	Stop a bad batch within one transaction	**Complete (also resolve, revoke, reinstate)**	contract `flagBatch`, `revokeBatch`; batch detail timeline in `pages/BatchDetail.tsx`
All state changes cryptographically signed	No server can forge an approval	**Complete**	MetaMask signs; the API reads the receipt and rejects any mismatch (`batchController.js`)
Verification in under five seconds (non-functional)	Smooth mobile scanning	**Met when warm**	Section 3. First request after a cold start measured 18 s; a fix is written and tested locally (`verifyController.js`, 1.5 s cap on the chain read)
Gasabo District, three drug categories	Realistic, focused pilot scope	**Complete**	15 Gasabo sectors in `mobile/lib/core/sectors.dart`; catalogue of antimalarials, antibiotics and ARVs in `backend/data`
Added: pack-level serial numbers and sale recording	Closes the cloned-QR weakness named in the proposal	**Complete**	`serialController.js`, `utils/gs1.js`; mobile "Mark as dispensed"
Added: hash-chained audit log, scan analytics, flag reports	Oversight and the evaluation metrics	**Complete**	`utils/audit.js`, `pages/AuditLog.tsx`, `pages/ScanAnalytics.tsx`
Pharmacist accounts	Sector-linked scans and sales	**Partial**	A regulator can create accounts; emailed temporary passwords and password reset are not built
Pilot with about 20 pharmacies and 200 users	Measure accuracy and usability	**Not run**	Not feasible in the build window. A seeded dataset (`scripts/seedDemoData.js`) and device testing stand in

Scope changes. Serial numbers and the audit chain were added because the proposal itself lists cloned QR codes and tamper evidence as risks. A public test network was not attempted because the proposal limits execution to local development environments; it is listed as future work.

2. Technical functionality

Architecture and data flow

The three tiers are decoupled. Presentation: Flutter app and React dashboard. Middleware: Express API with JWT authentication and role checks. Data: PostgreSQL (8 tables, versioned SQL migrations 001 to 007) plus the Solidity contract. The chain holds only addresses, timestamps, statuses and hashes; names, licences and flag reasons stay in the database and are anchored by hash.

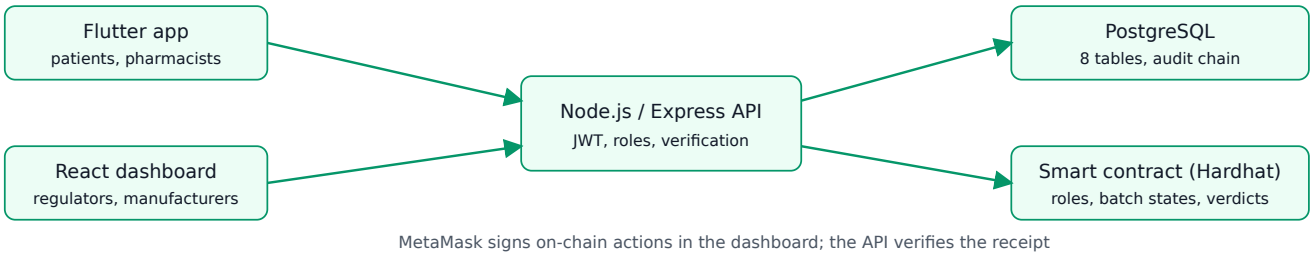


Figure 1: Three-tier architecture of the delivered system.

Core functions and the logic behind them

- **Client signs, server verifies.** For a new batch the API stores a draft and returns the exact values to sign. The browser wallet sends `submitBatch`. The API then reads the receipt and accepts it only if the event matches the batch, the metadata hash and the wallet. Approve, flag and revoke work the same way, and a flag reason is checked against the hash signed on-chain.
- **Scan verification.** The API parses the GS1 Digital Link (`/01/GTIN/10/batch/21/serial`), loads the batch and reads `verifyBatch` from the contract. The contract read is capped at 1.5 s. If the database and the chain disagree, the more severe result wins, so a flag on-chain cannot be hidden by a lagging database. The per-pack state then applies: a pack already sold returns "duplicate".
- **Manual entry.** A batch number alone is enough. Batch numbers are unique per manufacturer only, so if two products share one the API returns the candidates and the app asks the user to pick.
- **Public batch history.** The dashboard timeline is rebuilt from `batch_events` by replaying each event to compute the batch status after it. The public endpoint hides staff names, reasons and the manufacturer's submission step, and lists events oldest first.
- **Audit log.** Each entry stores the SHA-256 of its content and the previous hash. An advisory lock serialises writers so the chain cannot fork, and the dashboard can verify the whole chain.
- **Serial numbers.** 12 characters from a 32-symbol alphabet without look-alike characters (32^{12} combinations, no modulo bias).

Technical requirements

Frontend and backend: the dashboard calls a typed API client (`lib/api.ts`); the app uses an API service that turns network failures into readable messages. **Data handling:** parameterised SQL, transactions around every multi-table write, an idempotent catalogue import, and replay-safe confirm endpoints. **Logic:** role checks on every dashboard route, input validation on every field, and time handling fixed to Kigali time on the app.

3. Testing, validation and performance

Testing with different inputs

Input	Expected and observed behaviour
Approved, pending, flagged, revoked batches	Verdicts authentic, pending, flagged, revoked; source reads "blockchain + database"
Batch number only, in lower case	Found and normalised to upper case
Code printed with an old tunnel address	Still verifies, because the host is ignored
Unknown batch	"not found", never "authentic"
Malformed code, wrong GTIN check digit, blank batch	HTTP 400 with a clear message

Same batch number from two manufacturers	Candidate list returned; user chooses
Sale recorded without a token, or a pack sold twice	401 or 403; second sale returns "already dispensed"
Dashboard route without a token	Refused (401 or 403)

Testing strategies

Strategy	Why chosen	How conducted	Results	Improvement made
Unit (GS1)	Pure logic that every scan depends on	6 tests with the Node test runner: check digit, GTIN derivation, serial alphabet and uniqueness, link parsing	6 of 6 pass	Parser now rejects bad serials and ignores the host
Unit (smart contract)	Contract rules cannot be patched after deployment	17 Hardhat tests: roles, life cycle, flag and resolve, revoke, manufacturer revocation, pause, expiry (`test/PharmaTrace.test.js`)	Suite written to run with `npx hardhat test`	Confirmed that pause blocks submissions but not recalls
Integration	Bugs appear where API, database and chain meet	11 tests over HTTP against the seeded database and the contract; include authentication and audit-chain checks	11 of 11 pass (17 of 17 with the unit tests)	Fixed batch-only lookup and added the history endpoint
Device and usability	The scanner is used on low-end phones	Tecno Camon 15 over a mobile network, light and dark themes, patient and pharmacist flows	Found a crash from missing logo files, a layout overflow, a wrong server path (404) and a manual form that demanded the GTIN	Added a logo fallback, fixed the path, made the batch number the only required field
Performance	Proposal requires answers under five seconds	ngrok traffic inspector, per-request durations from the phone	See table below	Capped the blockchain read at 1.5 s

Performance and compatibility

Request (phone, through the free ngrok tunnel)	Duration
Request that does no work (404), repeated	0.28 to 0.66 s
Verify, warm	1.16 s
Batch history	0.94 s and 2.14 s
Verify, first request after the server started	18.26 s

Warm requests are well inside five seconds, and about half a second of each is the tunnel itself. The 18 s outlier came from the unbounded contract lookup and a cold database connection; the 1.5 s cap removes the first cause. Hardware: Tecno Camon 15 (low-end Android) for the app; Linux laptop for the API, dashboard and local chain; MetaMask for signing. **Limits:** no load or stress test was run, and compatibility was checked on one phone model and the browsers used during development.

4. Code quality and technical implementation

- **Separation of concerns.** Backend: routes map URLs, controllers hold rules, `utils` holds pure logic (GS1, audit chain) so it is testable alone, `config` holds connections. Dashboard: `pages` (screens), `components` (reusable parts), `lib` (API, wallet, types). App: `core`, `data`, `features`.
- **OOP in the app.** `DrugBatch` and `BatchHistory` models with factory constructors encapsulate parsing; `ThemeController` (a `ChangeNotifier`) and the `AppColors` theme extension give polymorphic theming; typed exceptions (`ApiException`, `AmbiguousBatchException`) separate error kinds; reusable widgets (`BrandLogo`, `SectorDropdown`, skeletons).
- **Algorithms.** GS1 mod-10 check digit; Digital Link parsing; unbiased serial generation; SHA-256 hash chain with advisory locking; status replay for the timeline; severity merge of database and chain; deterministic seeded random generator for repeatable demo data.

- **Maintainability.** Versioned migrations with a runner, a repeatable demo seed that writes real transactions, `.env` excluded, consistent naming, comments that explain reasons rather than syntax, and a README with install steps and the repository layout. The commit history records each feature and fix.

Limitations and future work

The rate limiter keeps counts in memory. Pharmacist password reset and emailed credentials are not built. The system runs on a local chain; a public test network, hosting in Rwanda and the 200-user pilot are the next steps.